Mobile landscape, Flutter, Dart I

Week 1 · Mobile Applications Development

B.Sc. Computer Science · Addis Ababa University

Lecture topics

1. Why mobile, why Flutter
2. How Flutter draws pixels
3. Dart, part I
4. The lab and the project

1 · Why mobile, why Flutter

The phone is the main computer

- More smartphones than PCs worldwide
- In Ethiopia, the phone is the **only** internet device for most users
- Any service you build will be used mostly on a phone

> Mobile development is the normal case, not a special case.

Three ways to build

Approach	Example	Trade-off
Native	Kotlin / Swift	Best access & performance; **two code bases**
Web / hybrid	PWA, Capacitor	One web app; WebView limits
Cross-platform compiled	**Flutter**, React Native	One code base, near-native

What Flutter is

- Google's open-source UI toolkit, first stable 2018
- Language: **Dart** → compiled to native ARM, JS or Wasm
- Draws **every pixel itself** (Impeller engine) — no OEM widgets
- Targets Android, iOS, web, Windows, macOS, Linux
- Hot reload: edit → see it in ~1 s

Why Flutter for this course

- One language, one code base → Android **and** iOS
- Consistent UI across devices — fewer "works on my phone" bugs
- Fast feedback loop keeps you learning, not waiting
- Used in production: Google Pay, BMW, Toyota, eBay Motors, and many Ethiopian fintech and delivery apps

Skills employers ask for

- Flutter **plus**
  - state management (Riverpod / BLoC) — *week 5*
  - testing — *weeks 6, 12*
  - REST and Firebase — *weeks 9–10*
  - CI/CD and releases — *weeks 12, 14*
  - responsible AI use — *week 13*

2 · How Flutter draws pixels

Everything is a widget

```dart
// lab/starter/lib/exercises/lecture_widget_example.dart
class TimetableScreen extends StatelessWidget {
  const TimetableScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Bus Timetable')),
      body: const Center(child: Text('Next bus to Sidist Kilo: 08:15')),
    );
  }
}
```

A widget is an **immutable description**. `build()` returns the subtree.

The widget tree

```
TimetableApp
└── MaterialApp
    └── TimetableScreen
        └── Scaffold
            ├── AppBar
            │   └── Text('Bus Timetable')
            └── Center
                └── Text('Next bus…')
```

Composition, not inheritance: you nest small widgets rather than subclass big ones.

From widgets to pixels

- You write the **description**; Flutter does the drawing
- Rebuilding a description is cheap, so Flutter rebuilds often
- **Week 2** shows the machinery behind this: the widget, element and render trees

Stateless vs Stateful (preview)

- `StatelessWidget` — output depends only on constructor arguments
- `StatefulWidget` — owns a `State` object that survives rebuilds and calls `setState()`

Week 2 goes through the full lifecycle.

Hot reload vs hot restart

	Hot reload `r`	**Hot restart `R`**
Speed	~1 s	~2–3 s
State kept?	**yes**	no
Re-runs `main()`?	no	yes
Class shape / enum changes	no	yes

PAUSE — PREDICT THE OUTPUT

What happens?

You change `'Welcome to Campus Companion'` to `'Welcome!'` and press **r**.

Then you change `runApp(const CampusCompanionApp())` to `runApp(const Placeholder())` and press **r**.

3 · Dart, part I

Why Dart?

Flutter needed a language that can

1. compile **ahead-of-time** to fast native code (release)
2. run in a **VM with hot reload** (development)
3. describe **UI trees** readably (named parameters, trailing commas)

Dart does all three. Feels like Java / Kotlin / TypeScript / Swift.

Hello

```dart
/// Doc comment — shows in IDE tooltips.
void main() {
  print('Hello, Dart!');   // in Flutter prefer debugPrint / a logger
}
```

Try it now: https://dartpad.dev

Variables

```dart
var city = 'Addis Ababa';  // inferred String
final year = 2026;         // set once, at runtime
const pi = 3.14159;        // compile-time constant
String name = 'Bet';       // explicit type
int? maybeAge;             // nullable
```

Guideline: `final` by default, `const` when the value is known at compile time, `var` only when you must reassign.

Types

- `int`, `double` (both `num`), `String`, `bool`
- `List`, `Set`, `Map`, `Record` — `(int, String)`
- Everything is an object; no primitives
- Static types with inference: `var x = 1;` is an `int` forever

```
print(10 / 4);   // 2.5   (double)
print(10 ~/ 4);  // 2     (int)
```

Null safety

```
String? nickname;                    // null by default
print(nickname?.length);             // → null, no crash
print(nickname ?? 'no nickname');    // default
nickname ??= 'Bet';                  // assign if null
nickname!.length;                    // assumes non-null — throws if null
```

The compiler knows which values can be null. A null error is a **compile error** unless you use `!`.

PAUSE — PREDICT THE OUTPUT

What prints?

```
void main() {
  int? a;
  final b = a ?? 5;
  a ??= 10;
  print('$a $b');
}
```

Operators you will use daily

```
final list = [1, 2, ...[3, 4]];          // spread → [1, 2, 3, 4]
final maybe = [1, ...?nullableList];     // null-aware spread

final label = score > 50 ? 'pass' : 'fail';
if (value is int) print(value + 1);      // type test promotes value
```

Control flow: the familiar

```
for (var i = 0; i < 3; i++) { ... }
for (final item in items) { ... }
while (running) { ... }
if (x > 0) { ... } else if (x < 0) { ... } else { ... }
```

Same as Java / C.

Control flow: switch expressions (Dart 3)

```dart
// lab/starter/lib/exercises/lecture_examples.dart
return switch (score) {
  >= 90 => 'A',
  >= 80 => 'B',
  >= 70 => 'C',
  >= 60 => 'D',
  _     => 'F',
};
```

Exhaustive, returns a value, no `break`. Prefer this over if-chains when mapping a value to another.

Patterns in switch

```dart
// lab/starter/lib/exercises/lecture_examples.dart
String describe(Object? value) {
  return switch (value) {
    null                    => 'nothing',
    final int n             => 'the integer $n',
    final String s          => 'the text "$s"',
    final List<Object?> l   => 'a list of ${l.length} items',
    _                       => 'something else',
  };
}
```

Collection `if` and `for`

```
final menu = [
  'Home',
  if (isLoggedIn) 'Profile',
  for (final e in extras) e.toUpperCase(),
];
```

You will use this constantly to build widget lists conditionally.

Functions

```
int add(int a, int b) => a + b;                          // arrow body

String greet(String name, [String greeting = 'Hi']) =>  // optional positional
    '$greeting, $name';

double convert(double v, {bool toCelsius = false}) {   // named, with default
  return toCelsius ? (v - 32) * 5 / 9 : v * 9 / 5 + 32;
}
convert(37);  convert(98.6, toCelsius: true);
```

Named parameters are how **every Flutter widget** takes arguments.

Functions are values

```dart
// lab/starter/lib/exercises/lecture_examples.dart
int sumOfEvens(List<int> numbers) {
  return numbers.where((n) => n.isEven).fold(0, (sum, n) => sum + n);
}
```

`(n) => n.isEven` is an anonymous function passed as an argument. Flutter callbacks (`onPressed: () { ... }`) work in the same way.

Collections

```dart
final names = <String>['Abel', 'Sara'];        // List
final unique = {'a', 'b', 'a'};                // Set → {a, b}
final ages = {'Abel': 21, 'Sara': 22};         // Map

names.add('Kidus');
ages['Kidus'] = 20;
print(ages['Nobody']);                         // null, not a crash
```

Iterable methods

```dart
// lab/starter/lib/exercises/lecture_examples.dart
Map<String, int> wordCount(String sentence) {
  final counts = <String, int>{};
  for (final word in sentence.toLowerCase().split(RegExp(r'\s+'))) {
    if (word.isEmpty) continue;
    counts.update(word, (n) => n + 1, ifAbsent: () => 1);
  }
  return counts;
}
```

`map`, `where`, `fold`, `any`, `every`, `firstWhere`, `toList` — these seven cover most needs.

PAUSE — PREDICT THE OUTPUT

What is the type, and what prints?

```
final evens = [1, 2, 3, 4].where((n) => n.isEven);
print(evens);
print(evens.toList());
```

LIVE CODING

Live coding: from Hello World to a timetable app

The script is in the speaker notes.

Project anatomy

```
campus_companion/
├── lib/main.dart           runApp()
├── lib/app.dart            root widget (course convention)
├── test/                   flutter test
├── pubspec.yaml            deps, SDK constraint, assets
├── analysis_options.yaml   lint rules (strict in this course)
├── android/ ios/ web/      platform hosts
└── build/                  generated — never commit
```

Add a package: `flutter pub add <name>`, which resolves the version for you.

4 · The lab and the project

Lab topics

1. Install Flutter **3.47.7**; `flutter doctor` reports no problems (Android can wait)
2. Hello World; **predict, then try** hot reload and hot restart
3. The Campus Companion application shell (`lib/app.dart`)
4. Eight Dart exercises — one test written **before** the code
5. Check a saved assistant answer: predict, run, mark the errors
6. Exit task on paper, laptop closed

Two TA checkpoints: after task 3 and after task 5.

The group project

- Groups of **four** — formed by end of week 2
- Proposal due **week 3**: title, problem, users, 5–8 user stories
- Milestones after weeks 6 and 10; submission week 14; presentations week 15
- 30 % of your mark — see `project/README.md`

Start thinking now: which app would *you* want on your phone?

AI use in this course

- Each lab task has an **AI-use level**: 0 No AI · 1 Explain only · 2 Review · 3 Draft and verify
- Disclose: a three-line **AI note** in each lab; `AI_USAGE.md` at project milestones
- You must be able to **explain** what you submit — checkpoints and oral checks
- **No AI** in exit tasks and exams
- Never paste passwords, keys or personal data into an AI tool

Rules: `syllabus/policies.md`

Using an assistant while you are learning

- A lab task exists to make **you** practise
- Code that works is not the same as code you understand
- **Try first** · ask about **one** thing · ask for an **explanation**
- Afterwards: close it and write the function again yourself

Checking an answer: **predict → run → analyse → test → check every claim**

Before the next lecture

- Finish the lab if you have not; the week 2 starter project builds on it
- Read: Dart *Classes*, *Mixins*, *Asynchrony support* (dart.dev/language)
- Form your group of four

Questions?

Notes, lab and quiz: github.com/betsegawlemma/mobile-application-development-course